Chapter1:Introduction

Database System Concepts

University of Guilan

Yasaman Boreshban

Fall 2025

Outline

- What is a Database?
- Why not File Systems?
- Data vs. Information
- Operational Environment & Real-World Examples
- Problems of File Systems → Motivation for DBMS
- Data Redundancy (Limited & Extensive)
- DBMS Overview & Types (Relational, NoSQL, etc.)
- Database Languages (DDL, DML, SQL basics)
- Transactions & ACID Properties
- Database Tuning (Introduction)

Why Databases?

- Every software system needs data management
- Databases organize and store information efficiently
- Support for data access, update, and sharing
- Foundation for reliable, scalable applications

Database Applications Examples

- Enterprise Information
  - Sales: customers, products, purchases
  - Accounting: payments, receipts, assets
  - Human Resources: Information about employees, salaries, payroll taxes.
- Manufacturing: management of production, inventory, orders, supply chain.
- Banking and finance
  - customer information, accounts, loans, and banking transactions.
  - Credit card transactions
  - Finance: sales and purchases of financial instruments (e.g., stocks and bonds; storing real-time market data
- Universities: registration, grades

Source: Silberschatz et al., Database System Concepts, 7th Edition

Database Applications Examples (Cont.)

- Airlines: reservations, schedules
- Telecommunication: records of calls, texts, and data usage, generating monthly bills, maintaining balances on prepaid calling cards
- Web-based services
  - Online retailers: order tracking, customized recommendations
  - Online advertisements
- Document databases
- Navigation systems: For maintaining the locations of varies places of interest along with the exact routes of roads, train systems, buses, etc.

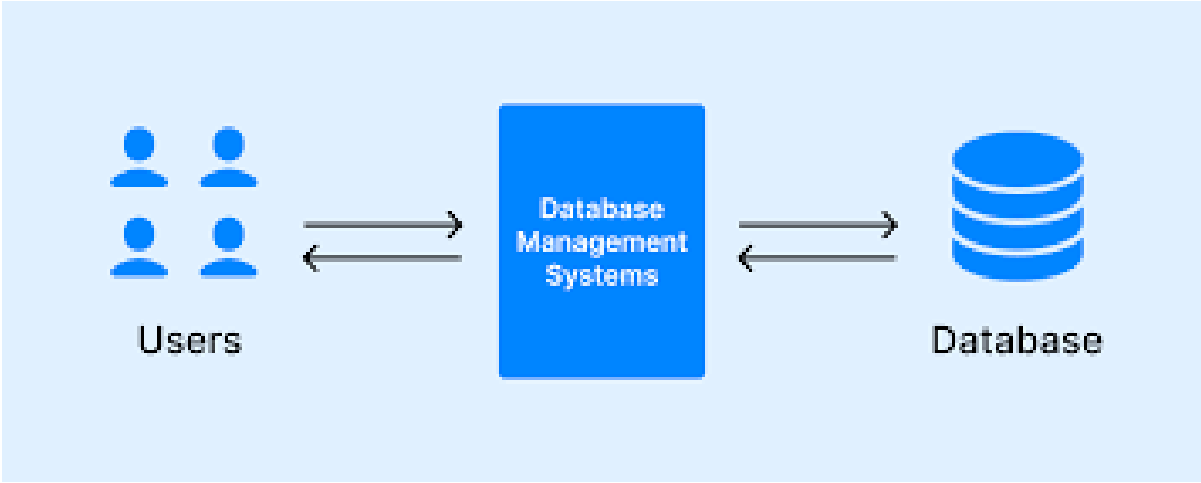
Source: Silberschatz et al., Database System Concepts, 7th Edition

Types of Data

- Structured Data
  - Organized in tables (rows & columns)
  - Example: Student records, bank accounts
- Semi-Structured Data
  - Not in fixed schema, but has tags or structure
  - Example: XML, JSON documents
- Unstructured Data
  - No predefined format
  - Example: Text documents, images, videos, emails

Database Management System (DBMS)

- Software to manage and control databases
- Provides interface between users/applications and data
- Hides details of data storage and file management
- Ensures data consistency, security, and recovery
- Examples: Oracle, SQL Server, MySQL, PostgreSQL



Types of DBMS

- Pre-relational Models
  - Hierarchical, Network
- Relational Model (RDBMS)
  - Based on tables (rows and columns)
  - Most widely used today
- Post-relational / NoSQL
  - Document-based, Key-Value, Graph
  - Flexible schema, scalable for big data

NoSQL Databases (Brief Overview)

- Not Only SQL → flexible alternatives to relational databases
  - Handle large-scale, unstructured, or semi-structured data
- Types:
  - Document-based (MongoDB)
  - Key-Value (Redis)
  - Columnar (Cassandra)
  - Graph (Neo4j)
  - Vector Databases (FAISS)
- Focus: scalability, flexibility, speed
- (Details covered in Advanced Database course)

Data vs Information

- **Data** = raw facts (numbers, text, symbols)
- **Information** = processed data with meaning

- Data: Raw facts and figures without context.
  - Example: 18
- Information: Processed data that has meaning in a specific context.
  - Example: 18 as a student GPA
- Data becomes information when it is interpreted and given meaning.
- Good database design helps transform data into useful information.

Database Definition & Features of DBMS

- Database: An organized collection of related data, stored and maintained systematically
  - Represents some aspects of the real world
  - Data are logically related
  - Designed for a specific purpose
- Key features of DBMS:
  - Persistent (data outlives the programs that use it)
  - Integrated (data is shared, consistent, and not isolated)
  - Based on a data model (e.g., relational, document, graph) Minimizes redundancy and inconsistency

Definition of Operational Environment

- Operational Environment = the real-world domain where data is generated and managed
- Defines the scope of the database system
- Examples:
  - University (students, courses, grades)
  - Bank (accounts, customers, transactions)
  - Hospital (patients, doctors, treatments)

Purpose of Database Systems

In the early days, database applications were built directly on top of file systems, which leads to:

- Data redundancy and inconsistency: data is stored in multiple file formats resulting induplication of information in different files
- Difficulty in accessing data
  - Need to write a new program to carry out each new task
- Data isolation
  - Multiple files and formats
- Integrity problems
  - Integrity constraints (e.g., account balance > 0) become "buried" in program code rather than being stated explicitly
  - Hard to add new constraints or change existing ones

Source: Silberschatz et al., Database System Concepts, 7th Edition

Purpose of Database Systems (Cont.)

- Atomicity of updates
  - Failures may leave database in an inconsistent state with partial updates carried out
  - Example: Transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
    - Ex: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- Security problems
  - Hard to provide user access to some, but not all, data

Database systems offer solutions to all the above problems

Source: Silberschatz et al., Database System Concepts, 7th Edition

Types of Data Redundancy

- **Data Redundancy** = repetition of data across multiple places
- **1. Limited Redundancy**
- **Natural:** repetition is unavoidable or logical
  - Example: course name repeated in students' enrollment records
- **Technical:** intentional redundancy to improve performance
  - Example: indexes, summary tables
- **2. Extensive Redundancy**
  - Caused by poor design or scattered systems
  - Leads to **inconsistency** and update anomalies
  - Example: student phone number stored separately in academic and financial systems

Database Languages in DBMS

- DDL (Data Definition Language)
  - define schema, tables, constraints
- DML (Data Manipulation Language)
  - insert, update, delete, retrieve data
- DCL (Data Control Language)
  - control access, permissions
- TCL (Transaction Control Language)
  - manage transactions (commit, rollback)

Transactions

- A transaction is a logical unit of database operations
- Consists of one or more database actions (e.g., read, write, update)
- Example: Transfer money from Account A to Account B
- Either all steps of a transaction are executed, or none

ACID Properties of Transactions

- Atomicity: All or nothing.
- Consistency: The database must remain in a valid state.
- Isolation: Transactions appear to run independently.
- Durability: Results of committed transactions persist even after failures.

Database Tuning

- Goal: Improve performance and efficiency of the DBMS
- Use indexing to speed up queries
- Optimize queries and schema design
- Manage storage and connections effectively

End of Chapter